

Name : Manish Ghanshani

Date : 23 / 05 / 26

- Existing code Block : Line 16

```
app.get("/users", (req, res) => {  
  const allUsers = users;  
  res.send(userList);  
});
```

Fixed Code :

```
app.get("/users", (req, res) => {  
  const allUsers = users;  
  res.send(allUsers);  
});
```

Bug : userList does not exist

- Existing code block : Line 23

```
const user = users.find(u => u.id === id);
```

Fixed Code :

```
const user = users.find(u => u.id === Number(id));
```

Bug : [req.params.id](#) is a string

- Existing code block : Line 27

```
function getUserById(id) {  
  const user = users.find(u => u.id === id);  
}
```

Fixed Code :

```
function getUserById(id) {  
  const user = users.find(u => u.id === id);  
  return user;  
}
```

```
}
```

Bug : No return statement

- Existing code block : Line 32

```
const total = notes.lenght;
```

Fixed Code :

```
const total = notes.length;
```

Bug : Spelling mistake

- Existing code block : Line 37

```
const data = fetchExternalData();
```

Fixed code :

```
const data = await fetchExternalData();
```

Bug : Function undefined

- Existing code block : Line 42

```
if (notes = []) {  
  console.log("No notes found");  
}
```

Fixed code :

```
if (notes.length === 0) {  
  console.log("No notes found");  
}
```

Bug : Use Assignment operator

- Existing code block : Line 52

```
const newId = generateNoteId;
```

Fixed code :

```
const newId = generateNoteId();
```

Bug : Function parenthesis not written

- Existing code block : Line 57

```
if (!title && !content) {  
    return res.send("Invalid input");  
}
```

Fixed code :

```
if (!title || !content) {  
    return res.send("Invalid input");  
}
```

Bug : Wrong operator used

- Existing code block : Line 61

```
const newNote = {  
    id: newId,  
    title: title,  
    content: content,  
    userId: userId  
};
```

Fixed code :

```
const newNote = {  
    id: generateNoteId(),  
    title: title,  
    content: content,  
    userId: userId  
};
```

Bug : Function explicitly made to create a note id

- Existing code block : Line 74

```
const noteIndex = notes.findIndex(n => n.id === id);
```

Fixed code :

```
const noteIndex = notes.findIndex(n => n.id === Number(id));
```

Bug : noteIndex is an integer whereas id is a string

- Existing code block : Line 85

```
user.name = username;
```

Fixed code :

```
user.name = name;
```

Bug : username undefined

- Existing code block : Line 92

```
const userNotes = notes.filter(n => n.userId === userId);
```

Fixed code :

```
const userNotes = notes.filter(n => n.userId === Number(userId));
```

Bug : Type mismatch

- Existing code block : Line 99

```
if (email === "admin@test.com" || password === "123456") {  
  res.send({ message: "Login successful" });  
} else {  
  res.send({ message: "Invalid credentials" });  
}
```

Fixed block :

```
if (email === "admin@test.com" && password === "123456") {  
  res.send({ message: "Login successful" });  
} else {  
  res.send({ message: "Invalid credentials" });  
}
```

Bug : It requires both validation to login successful

- Existing code block : Line 108

```
const user = users.filter(u => u.id === id);
```

Fixed code :

```
const user = users.find(u => u.id === id);
```

Bug : we need to find instead of filtering, filter returns an array

- Existing code block : Line 114

```
const total = a + b;
```

Fixed code :

```
const total = Number(a) + Number(b);
```

Bug : variables a and b are strings

- Existing code block : Line 118

```
app.listen(3000, () => {  
  console.log("Server running on port 5000");  
});
```

Fixed block :

```
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

Bug : App listens on port 3000

Fixed code :

```
const express = require("express");
const app = express();

app.use(express.json());

const users = [
  { id: 1, name: "Amit", email: "amit@test.com" },
  { id: 2, name: "Riya", email: "riya@test.com" }
];

const notes = [
  { id: 1, title: "Note 1", content: "Content 1", userId: 1 },
  { id: 2, title: "Note 2", content: "Content 2", userId: 2 }
];

app.get("/users", (req, res) => {
  const allUsers = users;
  res.send(allUsers);
});

app.get("/users/:id", (req, res) => {
  const id = req.params.id;
  const user = users.find(u => u.id === Number(id));
  res.send(user);
});

function getUserById(id) {
  const user = users.find(u => u.id === id);
  return user;
}

app.get("/notes/count", (req, res) => {
  const total = notes.length;
  res.send({ total });
});

app.get("/external-data", async (req, res) => {
```

```
    const data = await fetchExternalData();
    res.send(data);
  });

app.get("/notes", (req, res) => {
  if (notes.length === 0) {
    console.log("No notes found");
  }
  res.send(notes);
});

function generateNoteId() {
  return Math.random() * 1000;
}

const newId = generateNoteId();

app.post("/notes", (req, res) => {
  const { title, content, userId } = req.body;

  if (!title || !content) {
    return res.send("Invalid input");
  }

  const newNote = {
    id: generateNoteId(),
    title: title,
    content: content,
    userId: userId
  };

  notes.push(newNote);
  res.send(newNote);
});

app.delete("/notes/:id", (req, res) => {
  const id = req.params.id;
  const noteIndex = notes.findIndex(n => n.id === Number(id));
  if (noteIndex === -1) {
    return res.send("Note not found");
  }
});
```

```
    }
    notes.splice(noteIndex, 1);
    res.send({ message: "Note deleted" });
  });

app.put("/users/:id", (req, res) => {
  const id = req.params.id;
  const { name } = req.body;

  const user = users.find(u => u.id === id);
  user.name = name;

  res.send(user);
});

app.get("/user-notes/:userId", (req, res) => {
  const userId = req.params.userId;
  const userNotes = notes.filter(n => n.userId === Number(userId));
  res.send(userNotes);
});

app.post("/login", (req, res) => {
  const { email, password } = req.body;

  if (email === "admin@test.com" && password === "123456") {
    res.send({ message: "Login successful" });
  } else {
    res.send({ message: "Invalid credentials" });
  }
});

app.get("/profile/:id", (req, res) => {
  const id = Number(req.params.id);
  const user = users.find(u => u.id === id);
  res.send(user.name);
});

app.post("/sum", (req, res) => {
  const { a, b } = req.body;
  const total = Number(a) + Number(b);
```



```
    res.send({ total });  
  });  
  
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```